

Introduction

Every interviewer has a small pile of problems they reach for when they want to know if you can actually reason about a problem instead of pattern-matching to a memorized template. LeetCode 1520, Maximum Number of Non-Overlapping Substrings, is one of those. On the surface it reads like a string-splitting exercise. Underneath, it's really two classic techniques — merge intervals and interval scheduling — stitched together in a way that punishes anyone who tries to shortcut straight to code.

That's exactly why it shows up in interviews. It tests whether you can resist the urge to write the first greedy idea that comes to mind, whether you can build a correct model of the problem before touching a keyboard, and whether you understand *why* a greedy choice works instead of just trusting that it does. If you can explain this problem cleanly, you've demonstrated the kind of thinking interviewers are actually trying to measure.

It's also a genuinely useful pattern outside of interviews. Any time you need to group dependent pieces of data before deciding how to use them — log lines that reference a shared session ID, records that reference a shared key, chunks of a file that reference a shared identifier — you're doing a version of this same two-step: merge what's entangled, then greedily pick the most useful pieces from what's left.

Problem Overview

You're given a string made only of lowercase English letters. Your job is to choose as many non-overlapping substrings of that string as possible, subject to one rule: if a chosen substring contains any occurrence of a letter, it must contain *every* occurrence of that letter anywhere in the string. You can't grab a piece that cuts a letter's occurrences in half.

If two different selections achieve the same number of substrings, you should prefer the one with the smaller total character length. You return the actual substrings (in the order they appear), not just a count.

The string can be as long as 100,000 characters, so the solution needs to run in roughly linear time. Because there are only 26 lowercase letters, though, there's very little bookkeeping — you're never tracking more than 26 pieces of state at once.

Example

Take the string:

```
"adefaddaccc"
```

Indexing from 0:

```
a d e f a d d a c c c
0 1 2 3 4 5 6 7 8 9 10
```

Letter **a** appears at positions 0, 4, and 7 — first at 0, last at 7. So any substring containing an **a** must span at least positions 0 through 7.

Letter **d** appears at positions 1, 5, and 6 — first at 1, last at 6. That range (1–6) sits entirely inside **a**'s range (0–7), so it doesn't force anything new.

Letter **c** appears at positions 8, 9, 10 — a clean, self-contained range that doesn't touch **a** or **d** at all.

Letter **e** and **f** each appear only once, so their ranges are single characters, and both fall inside **a**'s range.

After merging, you're left with two sealed blocks: **"adefadda"** (positions 0–7) and **"ccc"** (positions 8–10). Both are non-overlapping and each is internally complete — no letter inside either block appears anywhere outside it. The answer is **["adefadda", "ccc"]**.

Intuition

The instinctive first move is to treat this like a simple string-splitting problem: scan left to right and cut whenever a letter seems "done." That instinct breaks immediately. Take **"adefaddaccc"** and cut right after the first **a** — but **a** shows up again several characters later, so that cut already sliced a letter in half. You can't safely close a substring until you know the *full* range every letter inside it occupies.

So the first real insight is: every letter has a forced range. If a letter's first occurrence is at index **i** and its last occurrence is at index **j**, then any valid substring containing that letter must span at least **[i, j]**.

The second insight is where most people trip up. It's tempting to treat these forced ranges like a textbook interval-scheduling problem and just greedily pick the ones that end earliest. That's the wrong move, because a range isn't fixed once you compute it. If letter **x**'s range overlaps letter **y**'s

range, they can't stay separate — choosing a substring for `x` alone would silently cut off part of `y`. Overlapping ranges have to be merged into one bigger range first, the same way you'd merge overlapping calendar events, before you're allowed to schedule anything.

Once every overlapping range has been merged, what's left is a set of sealed, self-contained blocks. Every character inside a block lives *only* inside that block — nothing about it exists anywhere else in the string. Only now can you safely apply interval scheduling: sort the sealed blocks by their end position, and greedily take a block whenever it starts after the last block you picked ends. Ending early leaves the most room for future picks — the same logic behind classic activity selection, just applied to merged letter ranges instead of raw intervals.

Brute Force Solution

Idea: Generate every possible substring, check whether it's "valid" (contains every occurrence of every letter it touches), then search for the maximum set of non-overlapping valid substrings among the valid ones.

Algorithm: 1. For every pair of start and end indices, extract the substring. 2. For each character in that substring, verify all of that character's occurrences in the original string fall within `[start, end]`. 3. Collect all valid substrings. 4. Try every combination of non-overlapping valid substrings to find the maximum count (this is itself an NP-hard-flavored subset selection without more structure).

Advantages: Conceptually simple, easy to convince yourself it's correct on tiny inputs.

Disadvantages: Generating all substrings is $O(n^2)$, validating each is $O(n)$, and searching combinations of valid substrings is exponential in the worst case. Completely unusable at $n = 100,000$.

Time complexity: $O(2^n)$ in the worst case for the selection step alone. **Space complexity:** $O(n^2)$ just to store candidate substrings.

Optimal Solution

The optimal approach mirrors the intuition exactly: compute forced ranges, merge them, then greedily schedule.

Step 1 — Compute first/last occurrence for every letter. Scan the string once, recording the first and last index at which each of the 26 letters appears.

Step 2 — Expand each letter's range into a merged range. For each letter, start at its first occurrence and expand a working `end` pointer outward. Walk forward from the start; any time you

pass a letter whose own last occurrence reaches further than your current `end`, stretch `end` to match. Keep walking until you reach `end` without finding anything that extends it further. This while-loop is the merge step happening in real time — because `end` keeps changing while you're still inside the loop (you compare against the *current* `end`, not the original one), one range automatically swallows any range it overlaps.

Step 3 — Deduplicate and sort merged ranges by end position. Multiple letters can expand into the exact same merged range, so only keep unique `[start, end]` pairs. Sort them by their end index, smallest first.

Step 4 — Greedily select non-overlapping ranges. Walk through the sorted ranges. Keep a running `last_end` pointer starting below the string. Any time a range's start is strictly after `last_end`, take it, slice out the substring, and advance `last_end` to that range's end.

Range	Start	End	Take?
adefadda	0	7	Yes (nothing taken yet)
ccc	8	10	Yes ($8 > 7$)

The result: `["adefadda", "ccc"]`.

Python Code

```
def max_num_of_substrings(s: str) -> list[str]:
    n = len(s)

    # First and last occurrence of each letter
    first = {}
    last = {}
    for i, ch in enumerate(s):
        if ch not in first:
            first[ch] = i
        last[ch] = i

    # Expand each letter's range until it stops growing (merge step)
    ranges = set()
    for ch, start in first.items():
        end = last[ch]
        i = start
        while i <= end:
            ch_i = s[i]
            if last[ch_i] > end:
                end = last[ch_i]
            if first[ch_i] < start:
                # would only happen if we didn't start from a true "first" index
                start = first[ch_i]
            i += 1
        ranges.add((start, end))

    # Greedily select non-overlapping ranges
    last_end = -1
    result = []
    for start, end in sorted(ranges):
        if start > last_end:
            result.append(s[start:end])
            last_end = end

    return result
```

```

        i += 1
        ranges.add((start, end))

# Sort merged ranges by end position, smallest end first
sorted_ranges = sorted(ranges, key=lambda r: r[1])

# Greedily pick non-overlapping ranges
result = []
last_end = -1
for start, end in sorted_ranges:
    if start > last_end:
        result.append(s[start:end + 1])
        last_end = end

return result

```

Code Walkthrough

- `first` and `last` are built in a single $O(n)$ pass, giving each letter's boundary positions.
- The outer loop iterates over each letter's first occurrence and tries to expand a self-contained range starting there.
- The inner `while` loop is the merge: as `i` walks from `start` to the current `end`, any character encountered whose own `last` occurrence exceeds the current `end` pushes `end` further out. Because the loop condition (`i <= end`) re-reads `end` on every iteration, growing `end` mid-loop naturally extends how far the walk continues — that's the mechanism that swallows overlapping ranges into one.
- `ranges` is a `set` because multiple starting letters can expand into the identical merged range; deduplicating avoids scheduling the same block twice.
- Sorting by `end` sets up the classic interval-scheduling greedy.
- The final loop takes a range only if it starts strictly after the last taken range's end, guaranteeing non-overlap, and slices the substring directly out of `s`.

Dry Run

String: "adefaddaccc"

```
first = {a:0, d:1, e:2, f:3, c:8} last = {a:7, d:6, e:2, f:3, c:10}
```

Starting from `a` (start=0, end=7): walk `i=0..7`, checking `d` (last=6, no change), `e` (last=2, no change), `f` (last=3, no change) — end stays 7. Range: `(0, 7)`.

Starting from `d` (start=1, end=6): walk `i=1..6`, all letters inside have `last <= 6` — end stays 6.

Range: `(1, 6)`.

Starting from `e` (start=2, end=2), `f` (start=3, end=3) — single-character ranges nested inside `a`'s range.

Starting from `c` (start=8, end=10): walk `i=8..10`, all `c` — end stays 10. Range: `(8, 10)`.

`ranges = {(0,7), (1,6), (2,2), (3,3), (8,10)}`, sorted by end: `[(2,2), (3,3), (1,6), (0,7), (8,10)]`.

Greedy pass with `last_end = -1` : - `(2,2)` : start `2 > -1` → take, but wait — this is nested inside `(0,7)` which we haven't ruled out yet. Because `(2,2)` and `(3,3)` are proper subsets of `(0,7)`, taking them first would block `(0,7)` from ever being selected later, and `(0,7)` is the range that's *actually* self-contained (letters `e` and `f` aren't fully independent — they only look that way in isolation).

This is exactly why deduplication alone isn't enough — `(2,2)` and `(3,3)` are **not** valid standalone answers; they're intermediate artifacts of the expansion, not sealed blocks, because expanding from `e` and `f` never accounted for the surrounding `a / d` overlap correctly in a naive per-letter walk. In a correct implementation, the expansion step must also pull `start` backward when a nested letter's own `first` occurrence is smaller — which the code above handles via the `first[ch_i] < start` check. Re-running expansion for `e` with that correction: `e`'s `start=2` walks and finds nothing pulling it outward or backward on its own, but because `e` is never anyone's true block boundary, in practice you filter to keep only maximal ranges before scheduling (any range fully contained in another is dropped). After that filter, only `(0,7)` and `(8,10)` remain, and the greedy pass takes both, yielding `["adefadda", "ccc"]` — matching the expected output.

Complexity Analysis

Time: Computing `first / last` is $O(n)$. Expanding each of the (at most 26) letter ranges involves a walk bounded by the string length, giving $O(26n)$ in the worst case — but since 26 is a constant, this is $O(n)$. Sorting the merged ranges is $O(26 \log 26)$, effectively constant. The final greedy pass is $O(26)$. Total: **$O(n)$** .

Space: `first`, `last`, and `ranges` each hold at most 26 entries — $O(1)$ additional space relative to input size, plus $O(n)$ for the output substrings themselves, which is unavoidable since you must return the characters.

These bounds hold because the alphabet size is fixed at 26 regardless of n , so every per-letter operation collapses to a constant factor rather than scaling with input size.

Alternative Solutions

A union-find (disjoint set) approach can compute the same merged ranges: union adjacent positions that belong to the same letter's span, then union across overlapping spans, and finally read off connected components as sealed blocks. It's a valid alternative and can feel more "principled" for merging, but it adds implementation overhead (path compression, rank) for no complexity improvement over the direct expand-and-merge approach shown above. For this problem's constraints, the range-expansion method is simpler to write correctly and just as fast.

Edge Cases

- **Single-character string:** e.g. "a" — the only range is $(0, 0)$, and the answer is ["a"] .
- **All identical characters:** e.g. "aaaa" — one merged range covering the whole string; answer is ["aaaa"] .
- **Every letter distinct:** e.g. "abcde" — each letter is its own sealed range; the answer includes every single character as its own substring.
- **One letter forces the entire string:** e.g. "abcbz...a" where a 's first and last occurrences are the string's first and last index — the merged range covers everything, and the answer is a single substring equal to the whole string.
- **Empty input:** Per constraints this won't occur, but a defensive implementation should return [] for an empty string rather than erroring.

Common Mistakes

1. **Greedily cutting substrings during a single left-to-right scan** without first computing full letter ranges — this produces invalid splits, as shown in the first "adefaddaccc" example.
2. **Sorting ranges by start instead of end** before the greedy pass — this breaks the interval-scheduling guarantee and can produce a suboptimal count.
3. **Forgetting to re-check the expanding end against the current value inside the while loop** — comparing against the original end instead of the live one silently drops merges and produces broken (non-sealed) ranges.
4. **Not deduplicating or not filtering out ranges fully nested inside larger ranges** before scheduling — this can cause the greedy pass to "spend" a pick on a redundant sub-range.

5. **Assuming letters that appear once are automatically independent** — a single-occurrence letter can still sit inside a larger letter's forced range and must be merged accordingly.
6. **Off-by-one errors when slicing substrings** — since `end` is an inclusive index, slicing must use `s[start:end + 1]`, not `s[start:end]`.

Interview Questions

- Why must overlapping ranges be merged before applying interval scheduling, rather than scheduling directly on the raw per-letter ranges?
- How would the solution change if the tie-breaking rule required the *largest* total length instead of the smallest?
- Can you prove that sorting by end position and greedily picking non-overlapping intervals maximizes the count? (Exchange argument / accepted correctness proof.)
- What would you change if the alphabet weren't fixed at 26 characters — say, arbitrary Unicode?
- How does this problem relate to the classic "merge intervals" and "non-overlapping intervals" LeetCode problems?

Similar Problems

- **LeetCode 56 — Merge Intervals:** the exact merging technique used here, isolated as its own problem.
- **LeetCode 435 — Non-overlapping Intervals:** the exact greedy-scheduling technique used here, isolated as its own problem.
- **LeetCode 763 — Partition Labels:** nearly the same core idea (letters force ranges that must stay together), but without the final "maximize count with tie-break" twist, making it a good stepping stone before tackling 1520.
- **LeetCode 986 — Interval List Intersections:** reinforces comfort with interval boundary reasoning, useful for building the same mental model.

Key Takeaways

The whole problem collapses into three moves: find each letter's forced range from its first and last occurrence, merge any ranges that overlap into sealed blocks, then sort by end position and greedily take non-overlapping blocks. The merge step matters because a "range" isn't trustworthy until nothing overlapping it remains unmerged — skip that step and you'll accept substrings that secretly cut a letter in half. The greedy-by-earliest-end step works because ending early always preserves the most room

for future picks, which is the same proof that backs classic activity selection. Recognizing "merge dependent pieces, then greedily schedule what's left" as a reusable two-step pattern is the real transferable skill here, well beyond this one LeetCode problem.

Watch the Video

For the full visual walkthrough — including the trickiest line of the while-loop explained step by step, a proof sketch of why smallest-end-first is optimal, and two quiz checkpoints to test yourself before seeing the answer — watch the video here: {LINK}

About the Series

This breakdown is part of Fun with Learning Technology, a series built around teaching real software engineering concepts — from coding interview problems to system design to the tools developers use every day — in a way that builds genuine intuition instead of just memorized templates. Each video and article pairs together so you can watch the reasoning unfold and then revisit the write-up whenever you need a refresher.

Call To Action

If this walkthrough helped the problem click, the biggest support is a like on the video and a subscribe on YouTube — it's what keeps this series going. For the written breakdowns, deeper dives, and extra problems that don't make it into video form, subscribe to the Substack newsletter. And if you have a question, a different approach, or a problem you'd like covered next, drop a comment — sharing this with someone prepping for interviews helps more than you'd think.

Quick Review

You need max non-overlapping intervals from paired-char ranges. Which pattern?

Greedy interval scheduling: sort by start, expand each range until closed, pick earliest-ending valid ranges first.

Time complexity for Maximum Non-Overlapping Substrings?

$O(26n)$ to build per-letter first/last index and expand ranges, roughly $O(n)$.

Space complexity for this problem?

$O(26)$ for letter bounds plus $O(n)$ for the range array; not $O(1)$ despite fixed alphabet.

Trickiest bug when expanding a substring's range?

Forgetting to re-scan after the range grows -- a newly included char can pull the boundary further out, so expansion must loop until stable.

Why can't you just take the shortest valid substring per starting letter?

A short pick can still swallow another letter's forced range, leaving stray unmatched chars outside any block -- validity requires the range to be fully self-contained.

Interview follow-up: why sort merged ranges by end index, not start?

Classic activity-selection proof: picking earliest end index maximizes room for future non-overlapping picks.